Session 2: AI for Coding — Practical Workflows and Demos

Elira Kuka and Tanner Regan

The George Washington University

Plan for Today: hands-on workflows

1. Coding with Claude Code overview
   - Writing and debugging code from plain English *EK*
   - "Vibe coding": building a complete tool without writing a line of code *EK*
2. Data analysis demos with Stata
   - Stata example with Claude Code from terminal *EK*
   - Stata example with Claude Code from VS Code *TR*
3. Python in VS Code overview
   - libraries and environments *TR*
   - Jupyter extension *TR*
4. Data analysis demo with Python
   - Python example with Claude Code from VS Code *TR*

Coding with Claude Code overview

Writing Code from Plain English

- You describe the task; AI writes the code — in any language (Stata, R, Python, Julia)
- This works even if you do not know the language
- Key skill: writing a good prompt
  - Describe inputs and outputs concretely: “I have a panel dataset with vars X, Y, Z…”
  - Specify what you want verified: “run it and show me the first 5 rows of output”
  - Be explicit about conventions: “use ‘i.year‘ for year fixed effects in Stata”
- ▷ *Placeholder: side-by-side example of prompt → code*

Debugging Code with AI

- Paste the error message and ask: “what is wrong and how do I fix it?”
- With Claude Code in agentic mode: it sees the error itself and iterates automatically
- Particularly powerful for:
  - Obscure Stata/R errors with cryptic messages
  - Merge problems (wrong key, many-to-many, unmatched observations)
  - Output that looks right but is subtly wrong (off-by-one, wrong unit of observation)
- Important caveat: AI can fix the error while introducing a different bug
  - Always verify output against known benchmarks or summary statistics

“Vibe Coding”: Building Tools Without Writing Code

- Concept: describe what you want at a high level; AI writes, runs, and refines all the code
- You act as the product manager, not the programmer
- Examples relevant to economists:
  - “Build a script that downloads monthly CPI from FRED, merges with my wage data, and plots real wages by year”
  - “Create a do-file that runs my DiD specification with all the standard robustness checks and exports tables to LaTeX”
  - “Write a web scraper that pulls contract award data from USASpending.gov”
- When it works well; when to be cautious

Automating Repetitive Data Tasks

- Common tasks AI handles easily:
  - Cleaning and reshaping messy data (wide $\leftrightarrow$ long, renaming, recoding)
  - Labeling variables and values from a codebook
  - Merging multiple datasets on complex keys
  - Generating a full set of summary statistics or balance tables
- Practical workflow for cleaning:
  1. Show Claude the raw data structure (variable list, codebook, first rows)
  2. Describe the cleaning steps in plain English
  3. Review the output, ask for changes, iterate
- ▷ *Placeholder: example or screenshot*

Recap: Three Modes of Operation

- **Ask before edits** (safest):
  - Claude proposes each file edit before making it; you approve or reject
  - Best for: getting started, when you want full control, unfamiliar codebases
- **Plan mode** (recommended for most tasks):
  - Claude writes a full plan in Markdown; you review and approve before anything runs
  - Produces clean, commented code with an explicit audit trail
  - Can view and revisit session history
- **Auto mode** (most powerful, most risk):
  - Claude acts autonomously until the task is done
  - Use for well-defined tasks; always check the output

Demo: Writing a Data Cleaning Script (Ask Mode)

- ▷ *Demo slide*
- Task: take a raw dataset, produce a clean analysis file
- Steps we will walk through:
  1. Point Claude Code at the project folder
  2. Write a minimum working example in “Ask before edits” mode
  3. Claude proposes each change; we approve
  4. Verify output matches expectations
- What to watch for:
  - Does the code run without errors?
  - Do summary statistics match what we expect?
  - Any implicit assumptions we need to check?

Demo: Cleaning Script Continued (Plan Mode)

- ▷ *Demo slide*
- Now run the same task in “Plan mode”:
  1. Ask Claude to write a plan before touching any files
  2. Review the plan in Markdown; request changes to the approach
  3. Approve; Claude executes and produces a clean, commented script
  4. Compare output to Ask-mode version: same result, better code
- Reviewing session history: how to audit what Claude did
- Ask Claude to update README with what was done (good practice)

When to Trust AI Code — and How to Verify It

- AI code can look completely correct and be wrong in subtle ways
- Verification checklist:
  - Does the code run without errors? (necessary, not sufficient)
  - Do observation counts make sense at each merge/collapse step?
  - Do means and distributions match codebook or prior literature?
  - Can you reproduce a known result (e.g., a published Table 1)?
  - Ask Claude: “what assumptions did you make?” — it will often reveal them
- General rule: trust AI code for *structure*; verify AI code for *content*

Building a Replication Package with Claude Code

- Claude Code can read your entire project and produce a replication package:
  1. Point Claude at the project directory
  2. Ask it to trace the data pipeline from raw files to final tables
  3. Ask it to write a master script that runs everything in order
  4. Ask it to write a README documenting data sources, software, and steps
- This task would normally take days — Claude does it in minutes
- You still need to verify: does running the master script reproduce the tables?
- AEA replication standards: what the package needs to include

Data analysis demos with Stata

Using Stata with Claude Code

- ▷ *Demo slide — Elira*
- Claude Code works with Stata via the terminal or VS Code (Stata Enhanced extension)
- What you can ask Claude to do:
  - Write a do-file from a plain-English description of the analysis
  - Debug a Stata error (paste the error; Claude explains and fixes it)
  - Translate a complex cleaning routine into clean, commented Stata code
  - Export regression tables to LaTeX (`esttab`, `outreg2`)
- Example: [PLACEHOLDER — data analysis example to demonstrate]

Stata Demo: Data Analysis with Claude Code

- ▷ *Demo slide*
- Task: *[Placeholder: insert specific empirical example]*
- Walk-through:
  1. Describe the dataset and goal to Claude in plain English
  2. Claude writes the do-file; we run it
  3. Inspect output, ask for revisions
  4. Ask Claude to add comments and export a summary table

Python in VS Code overview

Why Python?

-

Python for Economists: Common Tasks

- ▷ *Placeholder: example or screenshot*
- Data analysis with `pandas`: merging, collapsing, reshaping — analogous to Stata
- Econometrics: `statsmodels`, `linearmodels` (panel data, IV)
- Data collection: `requests`, `BeautifulSoup` for scraping (Session 3)
- Figures: `matplotlib`, `seaborn` — publication-quality plots
- Claude Code can translate your Stata code to Python (or vice versa) — useful for:
  - Running analyses you cannot do in Stata
  - Working with large datasets where Python is faster

Python basics

Libraries and Environments

- ▷ *Demo slide — Tanner*
- Anaconda manages Python installations and packages
- Each project gets its own **environment**: a fixed set of packages and versions
  - Prevents conflicts between projects
  - Ensures others can reproduce your results exactly
- Ask Claude Code to create and activate an environment for you

Python Setup in VS Code

- ▷ *Demo slide — Tanner*
- Required extensions: Python, Jupyter
- Managing environments:
  - Each project gets its own environment (set of packages + versions)
  - Prevents conflicts between projects
  - Ask Claude Code to create and activate an environment for you
- Running code:
  - **Jupyter notebook** (`.ipynb`): interactive cells, inline output — good for exploration
  - **Jupyter interactive window**: run `.py` scripts cell-by-cell — best of both worlds
  - **Script** (`.py`): run the full pipeline — best for production code

Running Python Code with Jupyter

- Three modes in VS Code:
  - **Jupyter notebook** (`.ipynb`): interactive cells, inline output — good for exploration
  - **Interactive window**: run `.py` scripts cell-by-cell — best of both worlds
  - **Script** (`.py`): run the full pipeline — best for production code
- Claude Code can write, run, and debug Python in any of these modes

Data Analysis Demo with Python

Python Exercise

- Download data from Kigali City
- Download data from Alpha Earth
- Visualize source data
- Build a sample frame
- Visualize the sample frame

Download data from Kigali City

1. Wrote out the prompt ”Prompt data scraping for Kigali.md”
2. Claude generated the plan ”kigali-city-data-mutable-frost.md” (∼10 mins)
3. Ran it without edits (using Jupyter window). Two issues:
   - Claude added packages to the yml but didn’t update the conda environment. Chatting with Claude we identified this andfixed it. Told claude to update the project CLAUDE.md file so that it always: (1) updates the python environment with newly required packages, and (2) updates the .yml file so that the environment is perfectly replicable.
   - I had to leave for daycare pickup and the code was still running. - I told Claude ”the script is running. I need to close my laptop and go home. Can I pause the script and restart it without disrupting the download?” -CC: ”The script has no resume logic — if you stop and restart it, it will re-download everything from scratch, including layers already completed. The fix is simple: add a file-existe

Wrapping Up

Key Takeaways from Sessions 1 & 2

1. AI handles the coding mechanics — your job is to verify the output and drive the analysis
2. Use Plan mode for any non-trivial task; it gives you an audit trail and catches mistakes early
3. Git + Claude Code = version-controlled, reproducible research without learning Git commands
4. CLAUDE.md files carry your preferences across sessions so you do not repeat yourself
5. Replication packages are now a hours-long task, not a weeks-long one
6. The critical skill is *verification*: check outputs, not just that the code runs

Things to Try Before the Next Session

- Set up VS Code with Claude Code, Git, and Python (use the setup guide)
- Ask Claude to write a small data task in your preferred language — verify the output
- Initialize a Git repo for one of your current projects
- Write a CLAUDE.md file for that project
- Coming up in Session 3: Finding and Collecting Data
  - Identifying datasets, scraping, APIs, parsing PDFs at scale

Questions?

- Questions on anything from Sessions 1 or 2?
- What tasks in your own research would you most like to try this on?